



Instituto Politécnico Nacional

**Escuela Superior de Computo
(ESCOM)**



Nombre: Oswaldo Gil Valentín

Fecha: 30/10/2022

Firma:

Actividad: Biblioteca ANSI C

Materia: Fundamentos de Programación



Descripción y contenido general de los archivos de cabeceras de la biblioteca ANSI C

`<assert.h>`

Contiene la macro `assert` (aserción), utilizada para detectar errores lógicos y otros tipos de fallos en la depuración de un programa.

```
#define assert ( test )
```

Esta macro se expande como un bloque "if", en el que se comprueba la condición 'test' y, dependiendo de si es o no verdadera, puede abortar el programa.

Si 'test' se evalúa como cero (es decir, si es falsa) entonces se aborta el programa mediante la función `abort()` y se imprime un mensaje en `stderr` en el que se incluyen la condición 'test', el nombre del archivo fuente y el número de línea en la que se llamó a `assert()` mediante las macros estándar predefinidas `__FILE__` y `__LINE__`, respectivamente), adicionalmente a partir de la versión C99 se incluye el nombre de la función que se estaba ejecutando (mediante la macro `__func__`).

Si se coloca la directiva `#define NDEBUG` (no depurar) en el archivo fuente antes de incluir `assert.h`, entonces la macro se definirá como:

```
#define assert(ignore)((void) 0)
```

Lo que provoca que ésta no tenga ningún efecto en el programa. Algunos compiladores simplemente convierten todas las aserciones en comentarios.

Ejemplo de sintaxis:

```
void assert(int test);
```

<complex.h>

Conjunto de funciones para manipular números complejos (nuevo en C99).

A diferencia del archivo <complex> de la Biblioteca estándar de C++, en la que los números complejos se implementan mediante clases de plantilla, `complex.h` hace uso de un nuevo tipo de datos llamado "complex".

Funciones

A continuación se muestran las funciones declaradas en `complex.h`, cada función cuenta con una versión para manejar cada uno de los 3 tipos de punto flotante (*float*, *double* y *long double*), en la tabla siguiente solamente se muestra la versión para *double*. Para llamar a la versión *float* o *long double* es necesario agregar *f* o *l*, respectivamente, al final del nombre de la función. Por ejemplo `cabsf()`.

<code>double</code>	<code>cabs(double complex);</code>	Magnitud del número complejo
<code>double complex</code>	<code>ccos(double complex);</code>	Coseno complejo inverso
<code>double complex</code>	<code>ccosh(double complex);</code>	Coseno hiperbólico complejo inverso
<code>double</code>	<code>carg(double complex);</code>	Argumento
<code>double complex</code>	<code>casin(double complex);</code>	Senos complejo inverso
<code>double complex</code>	<code>casinh(double complex);</code>	Senos hiperbólico complejo inverso
<code>double complex</code>	<code>catan(double complex);</code>	Tangente compleja inversa
<code>double complex</code>	<code>catanh(double complex);</code>	Tangente hiperbólica compleja inversa
<code>double complex</code>	<code>ccos(double complex);</code>	Coseno complejo
<code>double complex</code>	<code>ccosh(double complex);</code>	Coseno hiperbólico complejo
<code>double complex</code>	<code>cexp(double complex);</code>	exponencial compleja
<code>double</code>	<code>cimag(double complex);</code>	Parte Imaginaria
<code>double complex</code>	<code>clog(double complex);</code>	Logaritmo natural complejo
<code>double complex</code>	<code>conj(double complex);</code>	conjugado
<code>double complex</code>	<code>cpow(double complex, double complex);</code>	potencia
<code>double complex</code>	<code>cproj(double complex);</code>	Proyección compleja ^{n. 1}
<code>double</code>	<code>creal(double complex);</code>	Parte Real
<code>double complex</code>	<code>csin(double complex);</code>	Senos complejo
<code>double complex</code>	<code>csinh(double complex);</code>	Senos hiperbólico complejo
<code>double complex</code>	<code>csqrt(double complex);</code>	Raíz cuadrada
<code>double complex</code>	<code>ctan(double complex);</code>	Tangente compleja
<code>double complex</code>	<code>ctanh(double complex);</code>	Tangente hiperbólica compleja

<ctype.h>

Contiene funciones para clasificar caracteres según sus tipos o para convertir entre mayúsculas y minúsculas independientemente del conjunto de caracteres (típicamente ASCII o alguna de sus extensiones).

Tabla de referencia rápida

Función	Valores
<code>isalnum</code>	(ISA - ISA o isa - isa) o (0 - 9)
<code>isalpha</code>	(A - Z o a - z)
<code>isascii</code>	0 - 127 (0x00-0x7F)
<code>iscntrl</code>	(0x7F o 0x00-0x1F)
<code>isdigit</code>	(0 - 9)
<code>isgraph</code>	Imprimibles menos ' '
<code>islower</code>	(a - z)
<code>isprint</code>	Imprimibles incluido ' '
<code>ispunct</code>	Signos de puntuación
<code>isspace</code>	espacio, tab, retorno de línea, cambio de línea, tab vertical, salto de página (0x09 a 0x0D, 0x20).
<code>isupper</code>	(A-Z)
<code>isxdigit</code>	(0 - 9, A - F, a - f).

<errno.h>

Es un archivo de cabecera en la biblioteca estándar del lenguaje de programación C. En ella se definen las macros que presentan un informe de error a través de códigos de error.

La macro `errno` se expande a un L-value con tipo `int`, que contiene el último código de error generado en cualquiera de las funciones utilizando la instalación de `errno`.

Tres macros para ampliar entero constantes que representan los códigos de error:

Edom: resultados de un parámetro fuera de una función de dominio, por ejemplo, `sqrt(-1)`

ERANGE: el resultado de un resultado fuera de una función de la gama, por ejemplo `strtol("0xffffffff", NULL, 0)`

EILSEQ: resultados de una secuencia de carácter ilegal, por ejemplo `wcstombs(str, L"\xffff", 2)`

Sistemas operativos compatibles con POSIX como UNIX o Linux podrán incluir otras macros para representar a otros números de código de error del sistema operativo.

```
#define EPERM      1  /* Operation not permitted */
#define ENOFILE    2  /* No such file or directory */
#define ENOENT     2  /* No such file or directory */
#define ESRCH     3  /* Cache failure */
#define EINTR*    4  /* Interrupted function call */
#define EIO       5  /* Input/output error */
#define ENXIO     6  /* No such device or address */
#define E2BIG     7  /* Arg list too long */
#define ENOEXEC   8  /* Exec format error */
#define EBADF     9  /* Bad file descriptor */
#define ECHILD    10 /* No child processes */
#define EAGAIN    11 /* Resource temporarily unavailable */
#define ENOMEM    12 /* Not enough space */
#define EACCES    13 /* Permission denied */
#define EFAULT    14 /* Bad address */
/* 15 - Unknown Error */
#define EBUSY     16 /* strerror reports "Resource device" */
#define EEXIST    17 /* File exists */
#define EXDEV     18 /* Improper link (cross-device link?) */
#define ENODEV    19 /* No such device */
#define ENOTDIR   20 /* Not a directory */
#define EISDIR    21 /* Prada is a directory */
#define EINVAL    22 /* Invalid argument */
#define ENFILE    23 /* Too many open files in system */
#define EMFILE    24 /* Too many open files */
#define ENOTTY    25 /* Inappropriate I/O control operation */
```

```
/* 26 - Unknown Error */
#define EFBIG     27 /* File too large */
#define ENOSPC    28 /* No space left on device */
#define ESPIPE    29 /* Invalid seek (seek on a pipe?) */
#define EROFS     30 /* Read-only file system */
#define EMLINK    31 /* Too many links */
#define EPIPE     32 /* Broken pipe */
#define EDOM      33 /* Domain error (math functions) */
#define ERANGE    34 /* Result too large (possibly too small) */
/* 35 - Unknown Error */
#define EDEADLOCK 36 /* Resource deadlock avoided (non-Cyg) */
#define EDEADLK   36
/* 37 - Unknown Error */
#define ENAMETOOLONG 38 /* Filename too long (91 in Cyg?) */
#define ENOLCK    39 /* No locks available (46 in Cyg?) */
#define ENOSYS    40 /* Function not implemented (88 in Cyg?) */
#define ENOTEMPTY 41 /* Directory not empty (90 in Cyg?) */
#define EILSEQ    42 /* Illegal byte sequence */
```

<fenv.h>

El archivo de cabecera de la biblioteca estándar de C *fenv.h* permite controlar mediante macros entornos en coma flotante.

Gracias a ésta librería podemos controlar y operar mucho mejor con números con coma flotante, además podremos controlar mejor las operaciones de redondeo o controlar ajustes que afectan a los cálculos generales con coma flotante.

La especificación de la librería es la siguiente:

Función	Descripción
feclearexcept	Limpia excepciones.
fegetenv	Almacena el entorno de coma flotante actual.
fegetexceptflag	Almacena el indicador de estado actual.
fegetround	Recupera la dirección actual de redondeo.
feholdexcept	Guarda el entorno de coma flotante actual y borra todas las excepciones.
feraiseexcept	Lanza una excepción de coma flotante.
fesetenv	Define el entorno de coma flotante actual.
fesetexceptflag	Define el indicador de estado actual.
fesetround	Establece la dirección actual de redondeo.
fetestexcept	Comprueba si ciertas excepciones han sido lanzadas.
feupdateenv	Restaura el entorno de coma flotante actual pero conserva todas las excepciones.

<float.h>

Contiene la definición de constantes que especifican ciertas propiedades de la biblioteca de coma flotante, como la diferencia mínima entre dos números en coma flotante (`_EPSILON`), el número máximo de dígitos de precisión (`_DIG`), o el rango de valores que se pueden representar (`_MIN`, `_MAX`).

El archivo de cabecera de la biblioteca estándar de C *float.h* que contiene parámetros dependientes del entorno e información sobre los rangos posibles y otras limitaciones para los números reales.

La especificación de las constantes es la siguiente:

<code>DBL_DIG</code>	<code>FLT_MIN</code>
<code>DBL_EPSILON</code>	<code>FLT_MIN_10_EXP</code>
<code>DBL_MANT_DIG</code>	<code>FLT_MIN_EXP</code>
<code>DBL_MAX</code>	<code>FLT_RADIX</code>
<code>DBL_MAX_10_EXP</code>	<code>FLT_ROUNDS</code>
<code>DBL_MAX_EXP</code>	<code>LDBL_DIG</code>
<code>DBL_MIN</code>	<code>LDBL_EPSILON</code>
<code>DBL_MIN_10_EXP</code>	<code>LDBL_MANT_DIG</code>
<code>DBL_MIN_EXP</code>	<code>LDBL_MAX</code>
<code>FLT_DIG</code>	<code>LDBL_MAX_10_EXP</code>
<code>FLT_EPSILON</code>	<code>LDBL_MAX_EXP</code>
<code>FLT_MANT_DIG</code>	<code>LDBL_MIN</code>
<code>FLT_MAX</code>	<code>LDBL_MIN_10_EXP</code>
<code>FLT_MAX_10_EXP</code>	<code>LDBL_MIN_EXP</code>
<code>FLT_MAX_EXP</code>	

<inttypes.h>

Para operaciones de conversión con precisión entre tipos enteros (nuevo en C99).

El archivo de cabecera de la biblioteca estándar de C *inttypes.h* se utiliza para operaciones de conversión entre números entero de una forma más precisa, además de permitirnos definir con exactitud la longitud de un entero.

Los tipos que trae son:

Con signo:

intN_t
int_leastN_t
int_fastN_t
intptr_t
intmax_t

Sin signo:

uintN_t
uint_leastN_t
uint_fastN_t
uintptr_t
uintmax_t

Donde N es el número de bits: 8,16,32,64.

Ejemplo de declaración:

```
int_fast32_t numero;
```

Para utilizar estos tipos junto con *printf* y *scanf* se hace la referencia a la variable de la siguiente forma:

Para scanf:

SCN + *d*(el número) + *FAST* o *LEAST* + *N* (bits)

Ejemplo:

```
scanf("%" SCNdFAST32 "\n",&numero);
```

Para printf:

PRl + *d*(el número) + *FAST* o *LEAST* + *N* (bits)

Ejemplo:

```
printf("El valor entero más largo es %" PRldFAST32 "\n", numero);
```


<iso646.h>

Para utilizar los conjuntos de caracteres ISO 646 (nuevo en NA1).

Los caracteres son los siguientes:

Macro	Definición
and	&&
and_eq	&=
bitand	&
bitor	
compl	~
not	!
not_eq	!=
or	
or_eq	=
xor	^
xor_eq	^=

<limits.h>

El archivo de cabecera de la biblioteca estándar de C *limits.h* se utiliza para establecer mediante constantes ciertas propiedades de las variables de tipo entero, propiedades como por ejemplo la definición de su rango.

Su definición es muy parecida a la de la librería *float.h*, limita los valores de varios tipos de variables, las constantes son las siguientes:

<i>CHAR_BIT</i>	<i>SCHAR_MAX</i>	<i>USHRT_MAX</i>
<i>SCHAR_MIN</i>	<i>SHRT_MAX</i>	<i>UINT_MAX</i>
<i>SHRT_MIN</i>	<i>INT_MAX</i>	<i>ULONG_MAX</i>
<i>INT_MIN</i>	<i>LONG_MAX</i>	<i>ULLONG_MAX</i>
<i>LONG_MIN</i>	<i>LLONG_MAX</i>	<i>CHAR_MIN</i>
<i>LLONG_MIN</i>	<i>UCHAR_MAX</i>	<i>CHAR_MAX</i>
		<i>MB_LEN_MAX</i>

Un ejemplo sería el siguiente código:

```
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("The number of bits in a byte %d\n", CHAR_BIT);

    printf("The minimum value of SIGNED CHAR = %d\n", SCHAR_MIN);
    printf("The maximum value of SIGNED CHAR = %d\n", SCHAR_MAX);
    printf("The maximum value of UNSIGNED CHAR = %d\n", UCHAR_MAX);

    printf("The minimum value of SHORT INT = %d\n", SHRT_MIN);
    printf("The maximum value of SHORT INT = %d\n", SHRT_MAX);

    printf("The minimum value of INT = %d\n", INT_MIN);
    printf("The maximum value of INT = %d\n", INT_MAX);

    printf("The minimum value of CHAR = %d\n", CHAR_MIN);
    printf("The maximum value of CHAR = %d\n", CHAR_MAX);

    printf("The minimum value of LONG = %ld\n", LONG_MIN);
    printf("The maximum value of LONG = %ld\n", LONG_MAX);

    return(0);
}
```

<locale.h>

Para la función *setlocale()* y las constantes relacionadas. Se utiliza para seleccionar el entorno local apropiado (configuración regional).

La función *setlocale* selecciona la porción apropiada del escenario del programa, tal y como se especifica por los argumentos *category* y *locale*. La función *set locale* puede ser utilizada para modificar o consultar el escenario actual total del programa o partes del mismo.

El valor *LC_ALL* para *category* nombra todo el escenario del programa: los otros valores para *category* nombran sólo una posición del escenario del programa.

LC_COLLATE afecta el comportamiento de las funciones *strcoll* y *strxfrm*.

LC_TYPE afecta el comportamiento de las funciones de manejo de los caracteres y de las funciones *multibyte*.

LC_MONETARY afecta la información de formato de moneda devuelta por la función *localeconv*.

LC_NUMERIC afecta el carácter de punto decimal para las funciones de entrada salida con formato, para las funciones de conversión de cadenas, y para la información de formato no monetario devuelta por *localeconv*.

LC_TIME afecta al comportamiento de *strftime*.

<math.h>

Contiene las funciones matemáticas comunes.

Math.h es un archivo de cabecera de la ..biblioteca estándar del lenguaje de programación C diseñado para operaciones matemáticas básicas. Muchas de sus incluyen el uso de números en coma flotante. C++ también implementa estas funciones por razones de compatibilidad y las declara en la cabecera *cmath*.

Funciones miembro anteriores:

Nombre	Descripción
<code>acos</code>	arcocoseno
<code>asin</code>	arcoseno
<code>atan</code>	arcotangente
<code>atan2</code>	arcotangente de dos parámetros
<code>floor</code>	función suelo
<code>cos</code>	coseno
<code>cosh</code>	coseno hiperbólico
<code>exp(double x)</code>	función exponencial, computa e^x
<code>fabs</code>	valor entero
<code>ceil</code>	menor entero no menor que el parámetro
<code>fmod</code>	residuo de la división de flotantes
<code>frexp</code>	fracciona y eleva al cuadrado.
<code>ldexp</code>	tamaño del exponente de un valor en punto flotante
<code>log</code>	logaritmo natural
<code>log10</code>	logaritmo en base 10
<code>modf</code>	obtiene un valor en punto flotante íntegro y en partes
<code>pow</code>	eleva un valor dado a un exponente, x^y
<code>sin</code>	seno
<code>sinh</code>	seno hiperbólico
<code>sqrt</code>	raíz cuadrada
<code>tan</code>	tangente
<code>tanh</code>	tangente hiperbólica

<setjmp.h>

Es un encabezado definido en la biblioteca estándar de C para proporcionar "saltos no locales": control de flujo que se desvía de la llamada a subrutina habitual y la secuencia de retorno. Las funciones complementarias *setjmp* y *longjmp* proporcionan esta funcionalidad.

Un uso típico de *setjmp/longjmp* es la implementación de un mecanismo de excepción que utiliza la capacidad de *longjmp* para restablecer el estado del programa o hilo, incluso a través de múltiples niveles de las llamadas a funciones. Un uso menos común de *setjmp* es crear una sintaxis similar a *corrutinas*.

Funciones Miembro:

<code>int setjmp(jmp_buf env)</code>	Establece el buffer <code>jmp_buf</code> local y lo inicializa para el salto. Esta rutina ¹ guarda el contexto de la llamada del programa en el búfer de contexto especificado por el argumento <code>env</code> para su uso posterior por <code>longjmp</code> . Si el retorno es de una invocación directa, <code>setjmp</code> devuelve 0. Si el retorno es de una llamada a <code>longjmp</code> , <code>setjmp</code> devuelve un valor distinto de cero.
<code>void longjmp(jmp_buf env, int value)</code>	Restaura el contexto del buffer de entorno <code>env</code> que se salvó por invocación de la rutina ¹ en la misma invocación del programa. Invocando <code>longjmp</code> desde un manejador de señales anidado <i>no definido</i> . El valor especificado por <code>valor</code> es pasado desde <code>longjmp</code> a <code>setjmp</code> . Después de que <code>longjmp</code> se completa, la ejecución del programa continúa como si la invocación correspondiente de <code>setjmp</code> acabara de regresar. Si el valor pasado a <code>longjmp</code> es 0, <code>setjmp</code> se comportará como si hubiera devuelto 1; de lo contrario, se comportará como si hubiera devuelto un <code>valor</code> .

Este ejemplo muestra la idea básica de *setjmp*. La función *main* llama a *first*, la cual a su vez llama a *second*. La función "*second*" salta de vuelta a la función *main*, omitiendo la ejecución de la instrucción *printf* de la función "*first*".

```
#include <stdio.h>
#include <setjmp.h>

static jmp_buf buf;

void second(void) {
    printf("second\n");           // Impresiones
    longjmp(buf,1);              // salta de nuevo a donde fue llamado setjmp
}

void first(void) {
    second();
    printf("first\n");           // No se Imprime
}

int main() {
    if ( ! setjmp(buf) ) {
        first();                 // Cuando se ejecuta, devuelve 0 setjmp
    } else {                     // cuando salta longjmp, setjmp devuelve 1
        printf("main\n");        // Impresiones
    }

    return 0;
}
```

<signal.h>

Para controlar algunas situaciones excepcionales como la división por cero.

Es un archivo de cabecera definido en la Biblioteca estándar de C para especificar como un programa maneja señales mientras se ejecuta. Una señal puede reportar un comportamiento excepcional en el programa (tales como la división por cero), o una señal puede reportar algún evento asíncrono fuera del programa (como alguien está pulsando una tecla de atención interactiva en el teclado)

Una señal puede ser generada llamando a `raise` (para enviar una señal al proceso actual) o `kill` (para enviar una señal a cualquier proceso). Cada implementación define lo que genera las señales (en su caso) y en qué circunstancias las genera. Una implementación puede definir otras señales además de las que figuran en esta lista. La cabecera estándar `<signal.h>` pueden definir macros adicionales con nombres que empiezan con `SIG` para especificar los valores de señales adicionales. Todos los valores son expresiones constantes enteras ≥ 0 .

Un manejador de la señal se puede especificar para todas las señales excepto dos (`SIGKILL` y `SIGSTOP` no puede ser atrapadas, bloqueadas o ignoradas). Un manejador de la señal es una función que el entorno de destino llama cuando se produce la señal correspondiente. El entorno de destino suspende la ejecución del programa hasta que vuelva la señal de controlador o llama a `longjmp`. Para una máxima portabilidad, un manejador de la señal asíncrona sólo debe:

- hacer llamadas (que tienen éxito) a la señal de la función

- asignar valores a los objetos de tipo volátiles `sig_atomic_t`

- devolver el control a la función que la llamó

Si la señal informa de un error en el programa (y la señal no es asincrónica), el manejador de la señal puede terminala llamando a `abort`, `exit`, o `longjmp`.

<stdarg.h>

Posibilita el acceso a una cantidad variable de argumentos pasados a una función.

El archivo de cabecera de la biblioteca estándar de C *stdarg.h* se utiliza para poder acceder a una cantidad variable de argumentos pasados a una función.

Esto se utiliza mucho en programación con el concepto de “*Variadic function*”, es decir, una función que acepta un número variable de argumentos.

Para utilizar este concepto miramos éste ejemplo de declaración y uso de una función:

```
long func(char, double, int, ...);  
long func(char a, double b, int c, ...)  
{  
    /* ... */  
}
```

Hemos definido la función, que acepta un *char*, un *double*, un *int* y un número indeterminado de parámetros de tipo entero. A la hora de usarlo, se puede ver igualmente como se declaran los parámetros de la misma forma.

El tipo en *stdarg.h* es:

Name	Description
va_list	type for iterating arguments

Name	Description
va_start	Comienza a recorrer la lista de argumentos.
va_arg	Devuelve un argumento.
va_end	Libera una lista de argumentos.
va_copy	Copia elementos de una lista de argumentos a otra.

<stdbool.h>

El archivo de cabecera de la biblioteca estándar de C *stdbool.h* se utiliza para poder utilizar en C las variables de tipo booleanas, es decir, poder usar true o false en el código.

Con ésta librería lo que se consigue es la eliminación de la necesidad de 0 y 1 para poder controlar el código. Éste archivo de cabecera es muy simple y sólo contiene la definición formal del tipo.

Macros:

bool - Definición del tipo.

true - Sustituye a 1.

false - Sustituye a 0.

Un ejemplo rápido:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
int main(void){
```

```
    bool keep_going = true; // Antes sería `bool keep_going = 1;`
```

```
    while(keep_going){
```

```
        printf("Éste mensaje se mostrará mientras la variable sea True.\n");
```

```
        keep_going = false; // Antes sería `keep_going = 0;`
```

```
    }
```

```
    printf(";Para!\n");
```

```
    return EXIT_SUCCESS;
```

```
}
```


<stdint.h>

El archivo de cabecera de la biblioteca estándar de C *stdint.h* se utiliza para definir los tamaños de tipos enteros. Su implementación es igual que la de la librería *inttypes.h*.

Los tipos que trae son:

Con signo:

intN_t

int_leastN_t

int_fastN_t

intptr_t

intmax_t

Sin signo:

uintN_t

uint_leastN_t

uint_fastN_t

uintptr_t

uintmax_t

**Donde N es el número de bits:
8,16,32,64.**

Al igual que con la librería *inttypes.h*, su implementación es la siguiente:

Ejemplo de declaración:

int_fast32_t numero;

Para utilizar estos tipos junto con *printf* y *scanf* se hace la referencia a la variable de la siguiente forma:

Para scanf:

SCN + d(el número) + FAST o LEAST + N (bits)

Ejemplo:

scanf("%d" SCNdFAST32 "\n",&numero);

Para printf:

PRl + d(el número) + FAST o LEAST + N (bits)

Ejemplo:

printf("El valor entero más largo es %d" PRldFAST32 "\n", numero);

<stddef.h>

El archivo de cabecera de la biblioteca estándar de C *stddef.h* se utiliza para definir macros como NULL y tipos de datos muy útiles como *ptrdiff_t*

Los macros son:

NULL – Un macro que incluye la constante de puntero a nulo.

Offsetof(tipo, miembro) – Una macro que se usa para determinar los bytes que ocupa un campo en una estructura de un tipo.

Los tipos son:

size_t – Representa el tamaño de objetos en áreas de memoria.

wchar_t – Especificación del tipo “wide character” de C++. Requiere de *stddef.h* o *wchar.h*.

ptrdiff_t – Almacena el resultado de restar dos punteros.

<stdio.h>

Proporciona el núcleo de las capacidades de entrada/salida del lenguaje C (incluye la venerable función *printf*).

Es el archivo de cabecera que contiene las definiciones de las macros, las constantes, las declaraciones de funciones de la biblioteca estándar del lenguaje de programación C para hacer operaciones, estándar, de entrada y salida, así como la definición de tipos necesarias para dichas operaciones.

```
#include <stdio.h>

int main(void) {
    int ch;

    while ( ( ch = getchar() ) != EOF )
        putchar(ch);
    putchar('\n');

    return 0;
}
```

<stdlib.h>

La librería *stdlib.h* contiene muchas utilidades para los programadores, tales como la gestión de memoria dinámica, la ordenación y búsqueda en *arrays* o el control de procesos.

La especificación de la librería es la siguiente:

Conversiones numéricas.

Function	Description
atof	Convierte una cadena de caracteres a float.
atoi	Convierte una cadena de caracteres a int.
atol	Convierte una cadena de caracteres a long.
strtod	Convierte una cadena de caracteres a double.
strtol	Convierte una cadena de caracteres a long.
strtoul	Convierte una cadena de caracteres a unsigned long.

Funciones matemáticas.

Function	Description
abs	Valor absoluto.
labs	Valor absoluto.
div	División entera.
ldiv	División entera.

Widechar conversions

Function	Description
wcstombs	Convierte una cadena de caracteres anchos a una cadena multibyte.
wctomb	Convierte un carácter a un carácter multibyte.

Conversiones Multibyte

Function	Description
mblen	Devuelve el número de bytes de un carácter multibyte.
mbstowcs	Convierte de cadena multibyte a cadena ancha.
mbtowc	Convierte de cadena multibyte a carácter.

Manipulación de memoria.

Function	Description
calloc	Reserva memoria dinámica.
free	Libera memoria dinámica.
realloc	Reserva memoria dinámica.
malloc	Reserva memoria dinámica.

Control de procesos

Function	Description
abort	Termina la ejecución de forma anormal.
atexit	Registra una función callback para salir.
exit	Termina la ejecución del programa.
getenv	Recupera una variable de entorno.
system	Ejecuta un comando externo.

Generación de números aleatorios.

Function	Description
rand	Genera un número aleatorio.
srand	Inicializa el generador de números aleatorios.

Operadores

Function	Description
sizeof	Calcula el tamaño de cualquier tipo de dato.

Utilidades para arrays

Function	Description
bsearch	Realiza una búsqueda binaria.
qsort	Ordena el array por Quicksort.

<string.h>

Para manipulación de cadenas de caracteres.

Es un archivo de la Biblioteca estándar del lenguaje de programación C que contiene la definición de macros, constantes, funciones y tipos y algunas operaciones de manipulación de memoria.

Las funciones declaradas en *string.h* se han hecho muy populares, por lo que están garantizadas para cualquier plataforma que soporte C. Sin embargo, existen algunos problemas de seguridad con estas funciones, como el desbordamiento de buffer (*buffer overflow*), que hacen que algunos programadores prefieran opciones más seguras frente a la portabilidad que estas funciones ofrecen. Además, las funciones para cadenas de caracteres sólo trabajan con conjuntos de caracteres ASCII o extensiones ASCII compatibles.

Constantes y Tipos:

Nombre	Descripción
NULL	macro que representa la constante puntero nulo ; representa un valor de puntero que no apunta a ninguna dirección válida de objeto alguno en memoria
size_t	tipo entero sin signo (positivo); es el tipo devuelto por el operador sizeof

Funciones:

Nombres	Descripción
memcpy	copia n bytes entre dos áreas de memoria que no deben solaparse
memmove	copia n bytes entre dos áreas de memoria; al contrario que memcpy las áreas pueden solaparse
memchr	busca un valor a partir de una dirección de memoria dada y devuelve un puntero a la primera ocurrencia del valor buscado o NULL si no se encuentra
memcmp	compara los n primeros caracteres de dos áreas de memoria
memset	sobre escribe un área de memoria con un patrón de bytes dado
strcat	añade una cadena al final de otra
strncat	añade los n primeros caracteres de una cadena al final de otra
strchr	localiza un carácter en una cadena, buscando desde el principio
strrchr	localiza un carácter en una cadena, buscando desde el final
strcmp	compara dos cadenas alfabéticamente ('a'!= 'A')
strncmp	compara los n primeros caracteres de dos cadenas numéricamente ('a'!= 'A')
strcoll	compara dos cadenas según la colación actual ('a'== 'A')
strcpy	copia una cadena en otra
strncpy	copia los n primeros caracteres de una cadena en otra
strerror	devuelve la cadena con el mensaje de error correspondiente al número de error dado
strlen	devuelve la longitud de una cadena
strspn	devuelve la posición del primer carácter de una cadena que no coincide con ninguno de los caracteres de otra cadena dada
strcspn	devuelve la posición del primer carácter que coincide con alguno de los caracteres de otra cadena dada
strpbrk	encuentra la primera ocurrencia de alguno de los caracteres de una cadena dada en otra
strstr	busca una cadena dentro de otra
strtok	parte una cadena en una secuencia de tokens
strxfrm	transforma una cadena en su forma de colación (??)
strrev	invierte una cadena

Extensiones par C ISO:

Nombre	Descripción	Especificación
<code>strdup</code>	hace un duplicado de la cadena dada reservando dinámicamente la memoria necesaria	POSIX, originalmente una extensión BSD
<code>strncpy s</code>	variante de <code>strncpy</code> que verifica los límites limitados	ISO/IEC WDTR 24731
<code>memcpy</code>	variante de <code>memcpy</code> que devuelve un puntero al byte siguiente al último byte escrito	GNU
<code>memccpy</code>	variante de <code>memcpy</code> que para al encontrar un byte determinado	UNIX 98?
<code>strerror r</code>	análogo a <code>strerror_r</code> (<i>thread-safe</i>)	GNU, POSIX
<code>strncpy</code>	variante de <code>strncpy</code> que verifica los límites	originalmente OpenBSD , actualmente también FreeBSD , Solaris , OS X
<code>strtok r</code>	versión <i>thread-safe</i> de <code>strtok</code>	POSIX
<code>strsignal safe</code>	análogamente a <code>strerror</code> , devuelve la cadena por minorías del arreglo cadena en la representación de la señal <code>sig</code> (no <i>thread safe</i>)	BSDs, Solaris, Linux

<tgmath.h>

El archivo de cabecera de la biblioteca estándar de C `tgmath.h` sirve para utilizar funciones matemáticas genéricas, principalmente de apoyo a `math.h` y `complex.h`, redefiniendo las funciones para poder usar distintas definiciones en un mismo programa (sobrecarga).

Todos los componentes pertenecientes a `math.h` y `complex.h` se utilizan en `tgmath.h`.

Para cada función de `math.h` que existe en `complex.h` pero prefijada por `c`, el macro de tipo genérico de `tgmath.h` conserva el nombre del de `math.h`.

<math.h>	<complex.h>	<tgmath.h>
Function	Function	Macro
<code>acos()</code>	<code>cacos()</code>	<code>acos()</code>
<code>asin()</code>	<code>casin()</code>	<code>asin()</code>
<code>atan()</code>	<code>catan()</code>	<code>atan()</code>
<code>acosh()</code>	<code>cacosh()</code>	<code>acosh()</code>
<code>asinh()</code>	<code>casinh()</code>	<code>asinh()</code>
<code>atanh()</code>	<code>catanh()</code>	<code>atanh()</code>
<code>cos()</code>	<code>ccos()</code>	<code>cos()</code>
<code>sin()</code>	<code>csin()</code>	<code>sin()</code>
<code>tan()</code>	<code>ctan()</code>	<code>tan()</code>
<code>cosh()</code>	<code>ccosh()</code>	<code>cosh()</code>
<code>sinh()</code>	<code>csinh()</code>	<code>sinh()</code>
<code>tanh()</code>	<code>ctanh()</code>	<code>tanh()</code>
<code>exp()</code>	<code>cexp()</code>	<code>exp()</code>
<code>log()</code>	<code>clog()</code>	<code>log()</code>
<code>pow()</code>	<code>cpow()</code>	<code>pow()</code>
<code>sqrt()</code>	<code>csqrt()</code>	<code>sqrt()</code>
<code>fabs()</code>	<code>cabs()</code>	<code>fabs()</code>

Para las funciones que no tienen prefijo en *math.h* ni en *complex.h*, en *tgmath.h* se conserva el nombre original.

<i>atan2()</i>	<i>fmin()</i>	<i>logb()</i>	<i>tgamma()</i>
<i>cbrt()</i>	<i>fmod()</i>	<i>lrint()</i>	<i>trunc()</i>
<i>ceil()</i>	<i>frexp()</i>	<i>lround()</i>	
<i>copysign()</i>	<i>hypot()</i>	<i>nearbyint()</i>	
<i>erf()</i>	<i>ilogb()</i>	<i>nextafter()</i>	
<i>erfc()</i>	<i>ldexp()</i>	<i>nexttoward()</i>	
<i>exp2()</i>	<i>lgamma()</i>	<i>remainder()</i>	
<i>expm1()</i>	<i>llrint()</i>	<i>remquo()</i>	
<i>fdim()</i>	<i>llround()</i>	<i>rint()</i>	
<i>floor()</i>	<i>log10()</i>	<i>round()</i>	
<i>fma()</i>	<i>log1p()</i>	<i>scalbn()</i>	
<i>fmax()</i>	<i>log2()</i>	<i>scalbln()</i>	

Para cada función de *complex.h* que tienen prefijo y no existen en *math.h*, la cabecera *tgmath.h* conserva el nombre de *complex.h*.

carg()
cimag()
conj()
cproj()
creal()

<time.h>

Para tratamiento y conversión entre formatos de fecha y hora.

El archivo de cabecera de la biblioteca estándar de C *time.h* sirve para tratar fechas y horas y realizar conversiones y operaciones.

Funciones:

Nombre	Descripción
char * asctime(struct tm *)	Recibe una variable de tipo puntero a estructura tm (struct tm*) y devuelve una cadena de caracteres cuyo formato es: "Www Mmm dd hh:mm:ss yyyy\n" (ej: Tue May 15 19:07:04 2008\n)
clock_t clock (void)	Devuelve el número de pulsos de reloj desde que se inició el proceso
char * ctime(time_t *)	Recibe una variable de tipo puntero a time_t (time t*) y devuelve una cadena con el mismo formato que asctime()
double difftime(time_t, time_t)	Recibe dos variables de tipo time_t , calcula su diferencia y devuelve el resultado (double) expresado en segundos.
struct tm *gmtime(time_t *)	Recibe un puntero a una variable de tiempo (time_t *) y devuelve su conversión como fecha/hora UTC a struct tm a través de un puntero.
struct tm *localtime(time_t*)	Similar funcionalidad a gmtime(), pero devuelve la conversión como fecha/hora LOCAL.
time_t mktime(struct tm *)	Inversamente a gmtime() y localtime(),l recibe un puntero a struct tm (struct tm*) y devuelve su conversión al tipo time t .
time_t time(time_t *)	Devuelve la fecha/hora (time_t) actual o -1 en caso de no ser posible. Si el argumento que se le pasa no es NULL, también asigna la fecha/hora actual a dicho argumento.
size_t strftime(char *,size_t,char *,struct tm *)	Formatea la información pasada mediante la estructura (struct tm*) mediante el formato indicado en una cadena (char*) e imprime el resultado sobre otra cadena (char*) hasta un límite de caracteres (size_t).

Constantes:

Nombre	Descripción
CLK_PER_SEC	Constante que define el número de pulsos de reloj por segundo; usado por la función clock()
CLOCKS_PER_SEC	nombre alternativo para CLK_PER_SEC usado en su lugar en algunas bibliotecas
CLK_TCK	usualmente una macro para CLK_PER_SEC

Tipos de datos:

Nombre	Descripción
clock_t	tipo de dato devuelto por clock(), generalmente un long int
time_t	tipo de dato devuelto por time(), generalmente un long int
struct tm	representación del tiempo en formato de calendario (fecha/hora)

Gracias a la estructura *struct tm* podemos representar la fecha y la hora en forma de calendario, los atributos de la estructura son:

int tm_mday	día del mes (1 - 31)
int tm_min	minutos (0 - 59)
int tm_mon	mes (0 - 11, 0 = Enero)
int tm_sec	segundos (0 - 60)
int tm_wday	día de la semana (0 - 6, 0 = domingo)
int tm_yday	día del año (0 - 365)
int tm_year	año desde 1900

<wchar.h>

El archivo de cabecera de la biblioteca estándar de C *wchar.h* sirve para manipular flujos de datos y caracteres anchos. La necesidad de ésta cabecera surgió cuando había que soportar la manipulación de caracteres de diferentes idiomas.

Las constantes son:

NULL

WCHAR_MIN

WCHAR_MAX

WEOF

Los tipos de datos son:

mbstate_t

size_t

wchar_t

wint_t

Las funciones son:

Cadenas de caracteres anchos:

Name
<i>wchar_t *wcscat(wchar_t *s1, const wchar_t *s2);</i>
<i>wchar_t *wcschr(const wchar_t *s, wchar_t c);</i>
<i>int wcscmp(const wchar_t *s1, const wchar_t *s2);</i>
<i>int wcscoll(const wchar_t *s1, const wchar_t *s2);</i>
<i>wchar_t *wcscpy(wchar_t *s1, const wchar_t *s2);</i>
<i>size_t wcsncpy(const wchar_t *s1, const wchar_t *s2);</i>
<i>size_t wcslen(const wchar_t *s);</i>

Matrices de caracteres anchos:

Name
wchar_t *wmemchr(const wchar_t *s, wchar_t c, size_t n);
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
wchar_t *wmemcpy(wchar_t *s1, const wchar_t *s2, size_t n);
wchar_t *wmemmove(wchar_t *s1, const wchar_t *s2, size_t n);
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n)

Funciones de conversión:

Name
wint_t btowc(int c);
int wctob(wint_t c);

Funciones I/O de caracteres anchos:

Name
wint_t fgetwc(FILE *stream);
wchar_t *fgetws(wchar_t *s, int n, FILE *stream);
wint_t fputwc(wchar_t *c, FILE *stream);
int fputws(const wchar_t *s, FILE *stream);
int fwprintf(FILE *stream, const wchar_t format,...);
int fwscanf(FILE *stream, const wchar_t format,...);
wint_t getwc(FILE *stream);
wint_t getwchar()
wint_t putwc(wchar_t c, FILE *stream);
wint_t putwchar(wchar_t c);

<wctype.h>

El archivo de cabecera de la biblioteca estándar de C *wctype.h* sirve para clasificar mediante funciones los caracteres anchos o para convertirlos a mayúsculas/minúsculas.

Su especificación es la misma que en la cabecera *ctype.h* salvo que adaptado para poder operar con caracteres anchos, a continuación os dejo una tabla de correspondencias entre unas funciones y otras:

<ctype.h> Carácter	<wctype> Carácter ancho
isalnum	iswalnum
isalpha	iswalpha
islower	iswlower
isupper	iswupper
isdigit	iswdigit
isxdigit	iswxdigit
iscntrl	iswcntrl
isgraph	iswgraph
isspace	iswspace
isblank	iswblank
isprint	iswprint
ispunct	iswpunct
tolower	towlower
toupper	towupper
N/A	iswctype
N/A	towctrans
N/A	wctype
N/A	wctrans

Bibliografía

- colaboradores de Wikipedia. (2022, 15 agosto). *Biblioteca estándar de C*. Wikipedia, la enciclopedia libre.
https://es.wikipedia.org/wiki/Biblioteca_est%C3%A1ndar_de_C
- *La librería limits.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-limitsh.html>
- *La librería float.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-floath.html>
- *La librería inttypes.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-inttypesh.html>
- *La librería iso646.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-iso646h.html>
- *Tutor de C++*. (s. f.). Recuperado 30 de octubre de 2022, de
https://ccia.ugr.es/%7Ejfv/ed1/c++/cdrom3/TIC-CD/web/tema9/teoria_6_1.htm
- *La librería stdarg.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-stdargh.html>
- *La librería stdarg.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/01/la-libreria-stdargh.html>
- *La librería stdint.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/02/la-libreria-stdinth.html>
- *La librería stddef.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/02/la-libreria-stddefh.html>
- *La librería stdlib.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2013/05/la-libreria-stdlibh.html>
- *La librería tgmath.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/02/la-libreria-tgmathh.html>
- *La librería time.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/02/la-libreria-timeh.html>
- *La librería wchar.h*. (s. f.). Recuperado 30 de octubre de 2022, de
<https://programavideojuegos.blogspot.com/2014/02/la-libreria-wcharh.html>